

Final Project

刘子意 2022202573



2025

AI & ML
for Data Scientists



数据处理优化



模型优化与交叉验证



因子重要性与预测结果

1. 细节优化

```
def extract_elevator_household(info_str):
    """提取梯数和户数"""
    if pd.isna(info_str) or str(info_str).strip() in ['', ' (空白)', '空白']:
        return np.nan, np.nan

    info_str = str(info_str).strip()

    # 中文数字映射
    chinese_numbers = {
        '一': 1, '二': 2, '两': 2, '三': 3, '四': 4, '五': 5,
        '六': 6, '七': 7, '八': 8, '九': 9, '十': 10,
        '十一': 11, '十二': 12, '十三': 13, '十四': 14, '十五': 15,
        '十六': 16, '十七': 17, '十八': 18, '十九': 19, '二十': 20,
        '二十一': 21, '二十二': 22, '二十三': 23, '二十四': 24, '二十五': 25,
        '二十六': 26, '二十七': 27, '二十八': 28, '二十九': 29, '三十': 30,
        '三十一': 31, '三十二': 32, '三十三': 33, '三十四': 34, '三十五': 35,
        '三十六': 36, '三十七': 37, '三十八': 38, '三十九': 39, '四十': 40,
        '四十一': 41, '四十二': 42, '四十三': 43, '四十四': 44, '四十五': 45,
        '四十六': 46, '四十七': 47, '四十八': 48, '四十九': 49, '五十': 50,
        '五十一': 51, '五十二': 52, '五十三': 53, '五十四': 54, '五十五': 55,
        '五十六': 56, '五十七': 57, '五十八': 58, '五十九': 59, '六十': 60
    }

    if time_match:
        build_year = int(time_match.group(1))
        # 计算房龄: 2025 - 建成时间
        if build_year != 0:
            return 2025 - build_year
```

2. 特征工程

```
df_new['物业费_绿化交互'] = df_new['物业费平均'] * df_new['绿化率']
df_new['物业费_容积交互'] = df_new['物业费平均'] * df_new['容积率']
df_new['楼层_电梯交互'] = df_new['总楼层'] * df_new['配备电梯']
df_new['南北通透'] = df_new['朝向_南'] * df_new['朝向_北']
```

3. 数值型分类变量处理

由于无序分类变量不应直接用数值代表分类，将这些变量转为字符串后进行编码

```
# 由于分类变量不应直接用数值代表分类，将这些变量转为字符串后进行编码
numeric_columns = ['城市', '区域', '板块', '区县', '板块_comm',
                    'for col in numeric_columns:
                        if col in Xp_train.columns:
                            Xp_train[col] = Xp_train[col].astype(str)
                        if col in Xp_test.columns:
                            Xp_test[col] = Xp_test[col].astype(str)
                        if col in Xr_train.columns:
                            Xr_train[col] = Xr_train[col].astype(str)
                        if col in Xr_test.columns:
                            Xr_test[col] = Xr_test[col].astype(str)']
```

4. 中文文本处理 (Word2Vec)

```
# 1. 文本预处理
processed_texts = chinese_text_preprocessing(
    df_train[text_column],
    custom_stopwords=custom_stop_words
)

# 2. 训练Word2Vec模型
w2v_model = Word2Vec(
    sentences=processed_texts,
    vector_size=vector_size,
    window=window,
    min_count=min_count,
    workers=4,
    seed=random_state,
    epochs=10
)

# 3. 获取文档向量
document_vectors = []
for words in processed_texts:
    if len(words) == 0:
        doc_vec = np.zeros(vector_size, dtype=np.float64)
    else:
        word_vecs = []
        for word in words:
            if word in w2v_model.wv:
                word_vecs.append(w2v_model.wv[word].astype(np.float64))
        if word_vecs:
            doc_vec = np.mean(word_vecs, axis=0)
        else:
            doc_vec = np.zeros(vector_size, dtype=np.float64)
    document_vectors.append(doc_vec)

document_vectors = np.array(document_vectors, dtype=np.float64)

# 4. KMeans主题聚类
kmeans = KMeans(
    n_clusters=n_topics,
    random_state=random_state,
    n_init=10
```

```
=====
处理Price训练集 - 训练主题模型
=====
开始训练集文本处理，处理 5 个文本列...
处理：客户反馈 - 生成 5 个有效主题
处理：周边配套 - 生成 5 个有效主题
处理：交通出行 - 生成 5 个有效主题
处理：户型介绍 - 生成 4 个有效主题
处理：核心卖点 - 生成 5 个有效主题
训练集处理完成！列数变化：53 -> 58

=====
处理Price测试集 - 使用训练好的模型
=====
开始测试集文本处理，处理 5 个文本列...
处理：客户反馈 - 未知词比例：0.0% - 完成
处理：周边配套 - 未知词比例：0.8% - 完成
处理：交通出行 - 未知词比例：0.6% - 完成
处理：户型介绍 - 未知词比例：1.8% - 完成
处理：核心卖点 - 未知词比例：1.5% - 完成
测试集处理完成！最终形状：(20775, 58)
```

1. 集成算法与深度学习

```
# Price模型
models_p = {
    'RandomForest_p': RandomForestRegressor(
        n_estimators=200, max_depth=15, min_samples_split=10,
        min_samples_leaf=4, random_state=42, n_jobs=-1,
        max_features='sqrt', max_samples=0.8
    ),
    'GradientBoosting_p': GradientBoostingRegressor(
        n_estimators=200, learning_rate=0.05, max_depth=6,
        min_samples_split=10, min_samples_leaf=4, random_state=42,
        subsample=0.8, validation_fraction=0.1, n_iter_no_change=10
    ),
    'XGBoost_p': XGBRegressor(
        n_estimators=200, learning_rate=0.05, max_depth=8,
        subsample=0.8, colsample_bytree=0.8, random_state=42, n_jobs=-1,
        min_child_weight=3, reg_alpha=0.1, reg_lambda=1.0, gamma=0.1
    ),
    'MLP_p': MLPRegressor(
        hidden_layer_sizes=(200, 100, 50), activation='relu', solver='adam',
        alpha=0.0001, learning_rate='adaptive', max_iter=2000,
        early_stopping=True, random_state=42, batch_size=256
    ),
    'LightGBM_p': lgb.LGBMRegressor(
```

```
# Price集成模型
voting_regressor_p = VotingRegressor(
    estimators=[
        ('rf', RandomForestRegressor(
            n_estimators=150, max_depth=12, min_samples_split=8,
            min_samples_leaf=3, random_state=42, n_jobs=1,
            max_features='sqrt'
        )),
        ('gb', GradientBoostingRegressor(
            n_estimators=150, learning_rate=0.07, max_depth=5,
            min_samples_split=8, min_samples_leaf=3, random_state=42,
            subsample=0.85
        )),
        ('xgb', XGBRegressor(
            n_estimators=150, learning_rate=0.07, max_depth=7,
            subsample=0.85, colsample_bytree=0.85, random_state=42, n_jobs=1,
            min_child_weight=2, reg_alpha=0.05, reg_lambda=0.5
        )),
        ('lgb', lgb.LGBMRegressor(
            n_estimators=150, learning_rate=0.07, max_depth=8,
            subsample=0.85, colsample_bytree=0.85, random_state=42,
            n_jobs=1, verbose=-1, num_leaves=50, min_child_samples=15
        ))
    ],
    weights=[1.2, 1.0, 1.2, 1.1],
    n_jobs=1
)
```

Price模型排名:

1. XGBoost_p: 0.1978
2. LightGBM_p: 0.2049
3. GradientBoosting_p: 0.2275
4. VotingRegressor_p: 0.2636
5. RandomForest_p: 0.4003

Rent模型排名:

1. XGBoost_r: 0.2154
2. LightGBM_r: 0.2192
3. GradientBoosting_r: 0.2367
4. VotingRegressor_r: 0.2610
5. RandomForest_r: 0.3750

集成模型评估

Price模型:

集成模型: VotingRegressor_p (RMSE: 0.2636)

最佳基础模型: XGBoost_p (RMSE: 0.1978)

最佳基础模型优于集成模型

Rent模型:

集成模型: VotingRegressor_r (RMSE: 0.2610)

最佳基础模型: XGBoost_r (RMSE: 0.2154)

最佳基础模型优于集成模型

2. 交叉验证

```
# 转换为numpy数组 (节省内存)
if hasattr(X, 'values'):
    X_original = X.values
elif hasattr(X, 'to_numpy'):
    X_original = X.to_numpy()
else:
    X_original = X
```

```
if hasattr(y, 'values'):
    y_original = y.values
elif hasattr(y, 'to_numpy'):
    y_original = y.to_numpy()
else:
    y_original = y
```

数据抽样 (加速)

```
if sample_frac is not None and 0 < sample_frac < 1:
    sample_size = int(len(X_original) * sample_frac)
    rng = np.random.RandomState(random_state)
    indices = rng.choice(len(X_original), sample_size, replace=False)
    indices.sort()
```

Price模型交叉验证

RandomForest_p	RMSE: 0.3714 (±0.0029)
GradientBoosting_p	RMSE: 0.2034 (±0.0028)
XGBoost_p	RMSE: 0.1760 (±0.0021)
MLP_p	RMSE: 0.1746 (±0.0009)
LightGBM_p	RMSE: 0.1810 (±0.0030)
VotingRegressor_p	RMSE: 0.2324 (±0.0021)

节省内存

加速

1. 因子重要性

```
# 数值特征映射
for feat in num_features:
    if any('\u4e00' <= char <= '\u9fff' for char in str(feat)):
        encoded_name = f"num_{hash(str(feat)) % 10000:04d}"
        display_mapping[encoded_name] = str(feat)
    else:
        display_mapping[str(feat)] = str(feat)
```

=====
Price模型 - 特征重要性分析
=====

最佳模型: XGBoost_p
分析方法: 树模型内置特征重要性

前20个最重要特征:

1. 建筑面积	: 0.2225
2. 容积率	: 0.0399
3. 停车位	: 0.0307
4. 交易权属[动迁安置房]	: 0.0306
5. 板块_comm[969.0]	: 0.0185
6. 周边配套_主题标签[2]	: 0.0170
7. 房屋总数	: 0.0165
8. coord_x	: 0.0145
9. 建筑结构[混合结构]	: 0.0121
10. 板块_comm[954.0]	: 0.0109
11. 板块[959.0]	: 0.0102
12. coord_y	: 0.0100
13. 燃气费平均	: 0.0099
14. 核心卖点_主题标签[2]	: 0.0084
15. 区域[121.0]	: 0.0084
16. 区县[21.0]	: 0.0082
17. 建筑结构[未知结构]	: 0.0081
18. 供暖[集中供暖/自采暖/无供暖]	: 0.0079
19. 配备电梯	: 0.0076
20. 城市[3]	: 0.0076

前20个特征占比: 50.0%

树模型:
内置因子重要性

其他模型:
排列重要性

2. 预测结果

选择最佳模型组合...

最佳Price模型: XGBoost_p (RMSE: 0.197761)

最佳Rent模型: XGBoost_r (RMSE: 0.215409)

开始测试集文本处理, 处理 5 个文本列...

处理: 客户反馈 - 未知词比例: 0.0% - 完成

处理: 周边配套 - 未知词比例: 1.6% - 完成

处理: 交通出行 - 未知词比例: 1.2% - 完成

处理: 户型介绍 - 未知词比例: 2.4% - 完成

处理: 核心卖点 - 未知词比例: 2.2% - 完成

测试集处理完成! 最终形状: (34017, 58)

Data Set	Metrics	In sample	Out of sample	Cross-validation	Kaggle Score
Price	RandomForest_p	0.3512	0.4003	0.3714	52.8
Price	GradientBoosting_p	0.1925	0.2275	0.2034	70
Price	XGBoost_p	0.1568	0.1978	0.1760	72.4
Price	MLP_p	0.1469	0.4041	0.1746	67.2
Price	LightGBM_p	0.1676	0.2049	0.1810	71.8
Price	VotingRegressor_p	0.2226	0.2636	0.2324	66.5
Rent	RandomForest_r	0.3228	0.3750	0.3405	52.8
Rent	GradientBoosting_r	0.2016	0.2367	0.2137	70
Rent	XGBoost_r	0.1747	0.2154	0.1964	72.4
Rent	MLP_r	0.1827	0.4422	0.2212	67.2
Rent	LightGBM_r	0.1852	0.2192	0.2003	71.8
Rent	VotingRegressor_r	0.2206	0.2610	0.2317	66.5